Data Structures & Algorithms

Session 4 – Linked Lists (Singly & Doubly)

Dr. Yatish Bathla,

Ph.D. (Information Technology), Obuda University, Budapest

What you will be able to do today

Explain what a dynamic data structure is, and why arrays are not always enough

Draw a singly linked list with nodes, references, head and null

Insert, delete and search in a singly linked list

Describe how a doubly linked list adds backward links

Compare the cost of list operations with array operations

Recap: arrays and their limits

Array strengths

Direct access by index in O(1)

Compact, simple memory layout

Easy to loop over

Array limits

Size is fixed when created

Insert or delete in the middle shifts many items

Resizing means copying everything

Can we have a structure that grows and shrinks one item at a time?

Dynamic structures grow and shrink

Fixed

Static structure

Size is decided in advance. An array of 100 holds 100 items, no more.

Grows

Dynamic structure

Memory is requested while the program runs, one item at a time.

Fits

Why it matters

Size unknown in advance: playlists, undo history, print jobs, the call stack.

A dynamic structure uses memory in proportion to the data it holds.

The idea of a linked list



Each node holds a value and a reference to the next node

head is the only entry point; the last node points to null

A list is a chain: lose the head and you lose the list.

Anatomy of a node

```
class Node {
    int data;        // the value stored
    Node next;       // reference to the next node

    Node(int data) {
        this.data = data;
        this.next = null;
    }
}
```

Key ideas

data: the value we store

next: a reference to another Node

A node sees only its neighbour, never the whole list

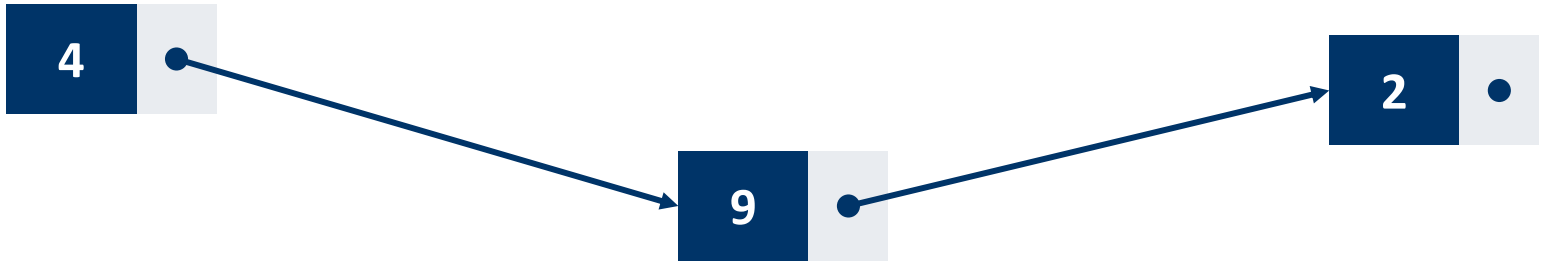
Generics (Node<E>) let it hold any type

References, not positions

Array: one block of memory, neighbours side by side



Linked list: nodes anywhere in memory, joined by references



Array: position links the items. List: references link the items.

Singly linked list: the operations

Operation	What it does	Cost
addFirst	New node becomes the head	O(1)
addLast	Append after the last node (O(1) if we keep a tail)	O(1) / O(n)
removeFirst	Head moves to the second node	O(1)
removeLast	Needs the second-to-last node	O(n)
find(key)	Walk until the key is found	O(n)
traverse	Visit every node once	O(n)

Work at the front of a singly linked list is cheap; work at the end is not.

Traversal: visiting every node

```
void display() {
    Node current = head;
    while (current != null) {
        System.out.print(current.data + " -> ");
        current = current.next;
    }
    System.out.println("null");
}
```

Key ideas

Start at head

Move with current = current.next

Stop when current is null

Never move head itself: you would lose the list

Cost: n steps, O(n)

Insert at the front

Before



After insertFirst(3)



1. new.next = head 2. head = new

Code: insertFirst and deleteFirst

```
class SinglyLinkedList {
    private Node head;    // null if empty

    void insertFirst(int value) {
        Node n = new Node(value);
        n.next = head;    // point to old first
        head = n;         // head moves to n
    }

    int deleteFirst() {   // list not empty
        int value = head.data;
        head = head.next;
        return value;
    }
}
```

Key ideas

insertFirst: 2 pointer changes

deleteFirst: 1 pointer change

Both O(1): nothing shifts

Check head == null before deleting

Java reclaims the unlinked node

Insert in the middle: four moves

1

Find

Walk to the node before the position.

2

Create

Make the new node and store the value.

3

Link forward

new.next = prev.next. Do this first.

4

Link back

prev.next = new. Order matters.

Reverse steps 3 and 4 and the rest of the list is lost.

Deleting a specific node

```
boolean delete(int key) {
    if (head == null) return false;
    if (head.data == key) {     // first node
        head = head.next;
        return true;
    }
    Node prev = head;
    while (prev.next != null) {
        if (prev.next.data == key) {
            prev.next = prev.next.next;
            return true;
        }
        prev = prev.next;
    }
    return false;               // not found
}
```

Key ideas

Handle empty and first-node cases first

prev stops one node before the target

prev.next = prev.next.next bypasses the node

Return false if the key is absent

Activity: think, pair, share

Problem

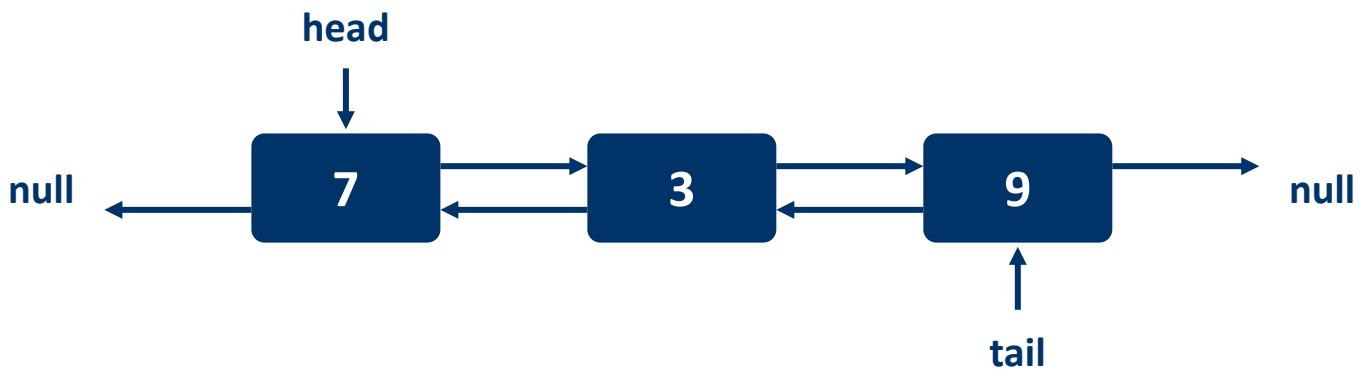
Start with an empty list. Run: insertFirst(5), insertFirst(8), insertLast(3), deleteFirst(), insertFirst(2). Draw the list after each step.

Linked list or array? Cost comparison

Operation	Array	Singly linked list
Access item at index i	O(1)	O(n)
Insert or delete at the front	O(n): shift items	O(1)
Insert at the end	O(1) if space remains	O(1) with tail, else O(n)
Delete from the end	O(1)	O(n)
Search for a value	O(n)	O(n)
Size	Fixed (or resize)	Grows and shrinks

Choose by workload: many front changes favour a list, many index reads favour an array.

Doubly linked list: two-way links



Each node stores data, prev and next

Walk forward with next and backward with prev

Two references per node cost memory but give movement in both directions.

Doubly linked: insert and delete

```
void insertFirst(int value) {
    DNode n = new DNode(value);
    n.next = head;
    if (head != null) head.prev = n;
    else tail = n;           // was empty
    head = n;
}

int deleteLast() {           // not empty
    int v = tail.data;
    if (head == tail) head = tail = null;
    else {
        tail = tail.prev;
        tail.next = null;
    }
    return v;
}
```

Key ideas

insertFirst fixes 3 links

deleteLast moves tail back one node

One-node list: head == tail

Both ends are O(1): no walking

Singly vs doubly linked lists

Feature	Singly	Doubly
Node stores	data, next	data, prev, next
Move backwards	No	Yes
Delete a known node	Needs the previous node	O(1) using the node itself
Memory per node	Less	More (one extra reference)
Typical use	Stacks, simple lists	Deques, undo history, browser tabs

Also see circular lists: the last node points back to the first (Goodrich 3.3).

Mini quiz

1. What does the next field of the last node hold?
2. Which is O(1) in a singly list: insertFirst or access by index?
3. In insertFirst, which line comes first: n.next = head or head = n?
4. What extra reference does a doubly linked node store?
5. Name one advantage and one cost of a doubly linked list.

Recap and check for understanding

A linked list is nodes joined by references; head is the entry point

Dynamic: grows and shrinks one node at a time, nothing shifts

Front operations are O(1); search and index access are O(n)

Doubly linked lists add prev: backward moves and easy deletion

Always test: empty list, first node, last node

Next session: stacks and LIFO processing, built on these lists